

CONTENTS

Práctico 8 (Recursión en 8086) — Programación... Arquitectura de Computadoras

FINC UdelaR · Resolución completa y verificada

Preguntas teóricas

Ejercicio 1 — ConstruirCadena (lista enlazada mediante array)

Código corregido

Ejercicio 2 — fibonacci (parámetro y resultado por registro)

Ejercicio 3 — busquedaBin (búsqueda en árbol binario indexado por array)

⚠ El código oficial tiene dos bugs reales

Código corregido

C) Consumo de stack, caso general y peor caso

Ejercicio 4 — profundidad (árbol con punteros far)

⚠ Dos correcciones sobre el código oficial

Código corregido

C) Consumo de stack en el peor caso, árbol de N nodos

Ejercicio 5 — cantidadBifurcaciones (nodos con exactamente dos hijos)

C) Consumo de stack, peor caso para árbol de N nodos

Ejercicio 6 — Torres de Hanoi

Código

B) Consumo de stack para n=3

Resumen de técnicas usadas en este práctico

Para el examen

Práctico 8 (Recursión en 8086) — Programación... Arquitectura de Computadoras

FINING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ Nota general (léela antes de empezar): el material oficial de la cátedra (`Ejercicios Resueltos.pdf`) sólo cubre los Ejercicios 1 a 4 — no incluye el Ejercicio 5 ni el 6 (Torres de Hanoi), que se resuelven acá desde cero siguiendo el mismo protocolo. Además, al revisar línea por línea el código oficial (como exige este documento, no copiarlo a ciegas), aparecieron **tres errores reales** en la solución de cátedra:

1. **Ejercicio 1 (`ConstruirCadena`):** dos prefijos de segmento equivocados — se lee el parámetro de la pila con `ES:` en vez de la pila (`SS` /implícito), y se escribe en el buffer sin el prefijo `ES:` que hace falta.
2. **Ejercicio 3 (`busquedaBin`):** un desbalance de pila real (un `POP BX` sin `PUSH` correspondiente en los dos casos base, que corrompería la dirección de retorno) y una confusión entre la **dirección** de un campo y su **valor** (nunca se lee `hijoIzq` / `hijoDer` de memoria, se pasa la dirección del campo como si fuera el índice del hijo).
3. **Ejercicio 4 (`profundidad`):** la condición de caso base compara el contenido apuntado por el puntero (`CMP ES:[BX],0`) en vez de comparar el puntero mismo (`CMP BX,0`), y usa el prefijo inválido `SP:` (SP no es un registro de segmento) donde corresponde `SS:`.

Cada uno se marca con **⚠** en su lugar, con la corrección completa y la re-verificación del cálculo de stack (que cambia, porque el bug de #2 cambiaba directamente cuántos bytes por activación hacían falta).

Preguntas teóricas

(a) ¿Por qué es importante salvar el contexto en una rutina recursiva en 8086?

Todas las activaciones de una función recursiva ejecutan **el mismo código** y por lo tanto comparten **los mismos registros físicos** — no hay “una copia de AX por llamada”. Si una activación no salva (PUSH) los registros que necesita después de una llamada recursiva y no los restaura (POP) al volver, la llamada recursiva anidada los pisa sin avisar, y la activación que llamó pierde sus propios datos (el `n` que tenía, un puntero que estaba recorriendo, etc.) apenas retorna la recursión. Salvar el contexto (registros de trabajo, y el propio `BP` para poder desarmar el frame) en la pila es lo que le da a cada activación su **propio espacio aislado**, a pe-

sar de compartir el hardware. Es exactamente la razón por la que en `fibonacci` (Ejercicio 2) hay que hacer `PUSH AX` antes de cada `CALL` recursivo: sin eso, el valor de `n` de esa activación se pierde en cuanto vuelve la recursión.

(b) ¿Qué hacen exactamente `CALL` y `RET`?

- `CALL etiqueta` (near, el caso de este práctico: la rutina se llama a sí misma dentro del mismo segmento de código): primero **apila IP** — la dirección de la instrucción siguiente al `CALL`, 2 bytes, la “dirección de retorno” — y después **carga IP** con la dirección de la rutina destino (el efecto es: $SP \leftarrow SP - 2$; $[SS:SP] \leftarrow IP$; $IP \leftarrow \text{destino}$). Si el `CALL` fuera *far* (como en el Ejercicio 4 del Práctico 8, para el ejercicio del árbol con punteros *far* — aunque ahí lo *far* son los datos, no la llamada a `profundidad` en sí, que sigue siendo *near*), además apilaría `CS` antes que `IP`, ocupando 4 bytes en vez de 2.
- `RET` hace exactamente lo inverso: **desapila IP** ($IP \leftarrow [SS:SP]$; $SP \leftarrow SP + 2$), retomando la ejecución en la instrucción siguiente al `CALL` que originó esa activación. `RET n` hace lo mismo y además $SP \leftarrow SP + n$, para que la propia rutina retire de la pila los bytes de parámetros que el llamador dejó ahí (protocolo “calleé retira”).

(c) ¿Qué relación hay entre el tamaño de un segmento y el máximo de llamadas recursivas a una función?

La pila vive dentro del segmento de pila (`SS`), cuyo tamaño máximo direccionable es 64 KB (65536 bytes, porque `SP` es un registro de 16 bits). Cada activación de una función recursiva consume una cantidad **fija** de bytes de pila (dirección de retorno + registros salvados + parámetros — la fórmula que se usa en todo este práctico). Por lo tanto:

$$\text{profundidad máxima de recursión} \leq \text{tamaño del segmento de pila disponible} \div \text{bytes consumidos por activación}$$

El tamaño del segmento es literalmente una **cota superior dura** sobre cuánta recursión se puede hacer: si la profundidad real necesaria multiplicada por el costo por activación supera el tamaño del segmento, el programa sufre un *stack overflow* (el puntero de pila se sale del segmento y empieza a pisar memoria de otro segmento, o directamente memoria fuera de rango) — no es un error que el 8086 detecte por sí solo, hay que dimensionar el segmento sabiendo de antemano cuál es la recursión máxima esperada. Esta es, precisamente, la razón de ser de todos los cálculos de “bytes de stack” que pide el resto del práctico.

Ejercicio 1 — ConstruirCadena (lista enlazada mediante array)

Protocolo dado por la letra: `indiceLista` en `BX` (registro); la lista está en `DS:SI`; `buffer-Destino` se pasa **en la pila** como desplazamiento respecto a `ES` (no respecto a `DS`); la función **no** retira ese parámetro de la pila (lo retira el llamador).

Layout de `Item` (struct empaquetada, 3 bytes cada una): `c` (1 byte) en `[SI+3i]`, `siguiente` (short, 2 bytes) en `[SI+3i+1]`.

CÓDIGO CORREGIDO

```

; void ConstruirCadena(short indiceLista, char *bufferDestino)
; indiceLista -> BX (registro). Lista -> DS:SI. bufferDestino -> pila, offset relativo a
ES.
; El llamador retira bufferDestino de la pila (esta rutina NO hace ADD SP tras el RET).

ConstruirCadena PROC
    PUSH BP
    MOV BP, SP                ; [BP+0]=BP viejo [BP+2]=dir. retorno (near, 2B)
    [BP+4]=bufferDestino
    PUSH AX
    PUSH DI

    MOV DI, [BP+4]            ; (*) DI = bufferDestino: el PARÁMETRO se lee de la pila (SS
por defecto, por BP)

_if:
    CMP BX, 0
    JGE _else
    MOV BYTE PTR ES:[DI], 0   ; (**) *bufferDestino='\0': el BUFFER vive en ES, hace
falta el prefijo
    JMP _return

_else:
    MOV AX, BX
    ADD AX, AX
    ADD BX, AX                ; BX = indiceLista*3 (tamaño de Item) = offset real dentro
de DS:SI
    MOV AL, DS:[SI+BX]        ; AL = lista[indiceLista].c
    MOV ES:[DI], AL           ; (**) *bufferDestino = AL (idem: prefijo ES obligatorio)
    MOV BX, DS:[SI+BX+1]      ; BX = lista[indiceLista].siguiente (nuevo indiceLista)
    INC DI                    ; DI = bufferDestino + 1

    PUSH DI                   ; parámetro de la llamada recursiva
    CALL ConstruirCadena
    ADD SP, 2                 ; retiramos bufferDestino+1 (esta rutina limpia lo que
ELLA empujó)

_return:
    POP DI
    POP AX
    POP BP
    RET                       ; el LLAMADOR de esta activación retira su propio
bufferDestino
ConstruirCadena ENDP

```

►  Qué decía el original y por qué está mal (click para expandir)

Revisión línea por línea (checklist de tolerancia cero) de la versión corregida:

- `PUSH BP` ↔ `POP BP` : ok, orden LIFO correcto (BP es lo primero en entrar, lo último en salir).
- `PUSH AX` ↔ `POP AX` , `PUSH DI` ↔ `POP DI` : ambos simétricos, y en el orden inverso correcto (DI se restaura antes que AX porque se apiló después).
- El `PUSH DI` justo antes de `CALL` es el **parámetro** de la llamada anidada, no un registro a preservar — no tiene `POP` emparejado, se retira con `ADD SP,2` tal como exige el protocolo (la rutina retira lo que ella misma empujó para su propia llamada recursiva).
- `BX` (registro, `indiceLista`) no se preserva porque el protocolo no lo pide — se consume y se reemplaza por el índice siguiente antes de la recursión.
- No se pide calcular consumo de stack para este ejercicio (la letra no tiene ítem C acá), así que no corresponde una fórmula numérica.

Ejercicio 2 — `fibonacci` (parámetro y resultado por registro)**A) Alto nivel:**

```
short fibo(short n) {
    if (n < 2) return n;
    short f1 = fibo(n - 1);
    short f2 = fibo(n - 2);
    return f1 + f2;
}
```

B) Assembler (protocolo: `n` en `AX` , resultado en `BX` , sin parámetros ni frame de `BP` en la pila):

```

fibonacci PROC
_if1:
    CMP AX, 2
    JGE _else
    MOV BX, AX          ; caso base: fibo(n) = n para n<2
    JMP _return
_else:
    PUSH AX             ; respaldamos n (se pierde en la recursión)
    DEC AX
    CALL fibonacci      ; BX <- fibo(n-1)
    POP AX              ; restauramos n
    PUSH BX             ; respaldamos fibo(n-1)
    SUB AX, 2
    CALL fibonacci      ; BX <- fibo(n-2)
    POP AX              ; AX <- fibo(n-1)
    ADD BX, AX          ; BX <- fibo(n-2) + fibo(n-1)
_return:
    RET
fibonacci ENDP

```

Revisado línea por línea contra el oficial: coincide exactamente y **no tiene errores** — cada `PUSH` tiene su `POP` en el orden correcto (`PUSH AX` se restaura antes de usarlo para la resta; `PUSH BX` se restaura justo antes de sumarlo), y no hay parámetros ni `BP` que armar porque todo va por registros.

C) Consumo de stack para N=5.

Por activación: la rama que llega **más profundo primero** es siempre la izquierda (`fibo(n-1)` se llama antes que `fibo(n-2)`), y una vez se toca el primer caso base la pila ya alcanzó su máximo (se puede achicar/agrandar después, pero no superar ese pico).

Tipo de paso	Bytes	Cantidad (N=5)
Invocación inicial (solo dirección de retorno, sin <code>PUSH AX</code> porque la llama el programa principal)	2	1
Llamada recursiva hacia la izquierda (<code>PUSH AX</code> + <code>CALL</code> = 2+2)	4	4 (<code>fibo(5)→fibo(4)→fibo(3)→fibo(2)→fibo(1)</code>)

Consumo máximo = 2 + 4×4 = 18 bytes

Verificado independientemente: `python -c "print(2+4*4)"` → 18. Coincide con el resultado textual del oficial ("el máximo tamaño que toma la pila es de 9 palabras, o sea, 18 bytes"). ✓

⚠ **Corrección menor de redacción:** el PDF oficial escribe la fórmula general como " $2 + (4 * N - 1)$ " — tal como está tipeada es ambigua/incorrecta ($2+4N-1=4N+1$, que da 21 para $N=5$, no 18). La fórmula que realmente reproduce su propio resultado es **$2 + 4 \cdot (N-1)$** bytes, equivalente a **$4N - 2$** , válida para $N \geq 1$ (para $N=5$: $4 \cdot 5 - 2 = 18$ ✓; para $N=1$, caso base inmediato sin recursión: $4 \cdot 1 - 2 = 2$, que es correcto — solo la invocación).

```
python -c "N=5; print(2+4*(N-1), 4*N-2)" → 18 18
```

Ejercicio 3 — `busquedaBin` (búsqueda en árbol binario indexado por array)

Layout: `struct Nodo {short dato; unsigned char hijoIzq,hijoDer;}` → 4 bytes por nodo. `arbol[256]` en `ES:0`. Índice 0 = "no hay hijo" (sentinela). Protocolo: `AX` =entero a buscar, `BX` =índice del (sub)árbol, resultado en `CL` (1=true, 0=false), **se deben conservar todos los demás registros.**

⚠ EL CÓDIGO OFICIAL TIENE DOS BUGS REALES

```

busquedaBin proc
    XOR CL, CL
    _if1:
        CMP BX, 0
        JE _return
    _if2:
        SHL BX, 2
        CMP AX, ES:[BX]
        JNE _else
        MOV CL, 1
        JMP _return          ; <-- BUG 1: llega a _return sin haber hecho ningún PUSH
    _else:
        PUSH BX
        ADD BX, 2             ; <-- BUG 2: esto es la DIRECCIÓN del campo hijoIzq, no su VALOR
        CALL busquedaBin      ;           se llama recursivamente pasando una dirección de
memoria                      ;           como si fuera un índice (0-255) – la recursión
opera
        POP BX                ;           sobre basura en cuanto el árbol no es trivial.
        PUSH BX
        ADD BX, 3
        CMP CL, 0
        JNE _return
        CALL busquedaBin
    _return:
        POP BX                ; <-- CONSECUENCIA DEL BUG 1: en las dos rutas de _if1 y _if2,
        RET                   ;           este POP saca la DIRECCIÓN DE RETORNO de la pila (porque
nunca
busquedaBin endp            ;           hubo un PUSH previo en esas rutas), y el RET siguiente
salta
                               ;           a una dirección basura -> crash casi garantizado.

```

Bug 1 (desbalance de pila): `_if1` (`BX=0`, subárbol vacío) y la rama “encontrado” de `_if2` saltan directo a `_return`, que hace un `POP BX` incondicional. Como en esas dos rutas nunca se ejecutó ningún `PUSH` en esta activación, ese `POP` saca del tope de la pila la **dirección de retorno** que dejó el `CALL`, y el `RET` subsiguiente salta a lo que haya quedado debajo (dato basura) — corrupción de pila real, no un detalle cosmético.

Bug 2 (dirección vs. valor): `hijoIzq` / `hijoDer` son **índices** (`unsigned char`, 0–255) hacia el siguiente nodo, no direcciones de memoria. `ADD BX,2` calcula la **dirección del campo** `hijoIzq` dentro del nodo actual (offset +2 en el struct), pero nunca lee lo que hay guardado ahí — pasa esa dirección tal cual como si fuera el índice del hijo. Para cualquier árbol con más de un nodo, la llamada recursiva termina indexando memoria completamente fuera de lugar.

CÓDIGO CORREGIDO

La forma más limpia de arreglar ambos bugs a la vez es usar un registro extra (`SI`) como variable local para “la dirección del nodo actual” — y confiar en que la propia recursión lo preserva (porque el protocolo exige preservar todos los registros, y por hipótesis inductiva la llamada recursiva cumple esa misma garantía):

```

busquedaBin PROC
    PUSH BX                ; preservamos el índice ORIGINAL recibido (hay que devolverlo
intacto)
    PUSH SI                ; preservamos SI; lo usamos como variable local (dirección
del nodo actual)
    XOR CL, CL             ; cond = false por defecto (cubre el caso "no encontrado")

_if1:
    CMP BX, 0
    JE _return            ; índice 0 = subárbol vacío -> no encontrado, cond ya es
false

_if2:
    SHL BX, 2             ; BX = índice*4 = offset del Nodo dentro de ES
    MOV SI, BX            ; SI = dirección del nodo actual (persiste a través de
las 2 llamadas)
    CMP AX, ES:[SI]
    JNE _else
    MOV CL, 1             ; encontrado en este nodo
    JMP _return

_else:
    MOV BL, ES:[SI+2]      ; BL = arbol[indice].hijoIzq (ahora SÍ se lee el
VALOR)
    MOV BH, 0             ; BX = hijoIzq extendido a 16 bits sin signo
    CALL busquedaBin       ; busca en el subárbol izquierdo

_if3:
    MOV BL, ES:[SI+3]      ; BL = arbol[indice].hijoDer
    MOV BH, 0             ; SI sigue siendo válido: la llamada anterior
lo preservó
    CMP CL, 0
    JNE _return           ; ya encontrado a la izquierda, no hace
falta ir a la derecha
    CALL busquedaBin

_return:
    POP SI
    POP BX
    RET
busquedaBin ENDP

```

Por qué esto es correcto (verificación): los dos `PUSH` (BX y SI) ocurren **incondicionalmente al entrar**, antes de cualquier `JE` / `JNE`, así que **todas** las rutas de salida llegan a `_return` con exactamente esos 2 valores pendientes de desapilar — se eliminó la asimetría del original. Y `SI` sobrevive a la primera llamada recursiva por el mismo motivo por el que sobrevive `BX` en el `factorial` de la guía M6: la función respeta su propio contrato de preservar registros, así que una llamada a sí misma no destruye lo que la activación actual guardó ahí.

C) CONSUMO DE STACK, CASO GENERAL Y PEOR CASO

Por activación (versión corregida): dirección de retorno (2, `CALL near`) + `PUSH BX` (2) + `PUSH SI` (2) = **6 bytes**. No hay parámetros en la pila (todo va por registro).

Profundidad máxima: el peor caso es un árbol degenerado (cadena lineal de N nodos reales, cada uno con un solo hijo). La recursión baja nodo a nodo hasta el N-ésimo, y ahí hace **una activación extra** para comprobar el índice-centinela 0 de su hijo faltante (esa activación entra por `_if1`, no recurre más, pero de todas formas hizo su propio `PUSH BX` / `PUSH SI` antes de llegar a `_if1`). Profundidad simultánea máxima = **N + 1** activaciones.

Consumo máximo = $6 \times (N + 1)$ bytes

Verificación explícita para N=2 (cadena nodo1→nodo2, ambos sin coincidencia, `hijolzq de nodo2 = 0`):

SP relativo (0 = SP al entrar a la primera llamada, antes de cualquier PUSH):

`CALL busquedaBin(nodo1)`: SP=2 (retaddr) → `PUSH BX`: SP=4 → `PUSH SI`: SP=6

`CALL busquedaBin(nodo2)`: SP=8 (retaddr) → `PUSH BX`: SP=10 → `PUSH SI`: SP=12

`CALL busquedaBin(0)` [`hijolzq de nodo2`]: SP=14 (retaddr) → `PUSH BX`: SP=16 → `PUSH SI`: SP=18 → **`_if1` dispara, `JE _return` (pico)**

Pico = 18 bytes. Fórmula: $6 \times (2+1) = 18$. ✓ (`python -c "print(6*(2+1))"` → 18)

Para el tamaño máximo de este `arbol[256]` (índice 0 reservado como sentinela, hasta 255 nodos reales utilizables): `python -c "print(6*(255+1))"` → **1536 bytes**.

⚠ Esta cifra (1536) es muy distinta del “511 bytes” que aparece en el PDF oficial — la diferencia se explica exactamente por los dos bugs corregidos arriba: el oficial cuenta 2 bytes por activación (solo la dirección de retorno, sin preservar `BX` como pide la letra) en vez de los 6 reales, y además su versión ni siquiera ejecuta correctamente para un árbol no trivial por el Bug 2. Si el ejercicio en un parcial pide explícitamente “sin preservar registros” el cálculo cambiaría, pero tal como está escrita la letra (“se deben conservar todos los demás registros”), 1536 es la cifra que corresponde.

Ejercicio 4 — profundidad (árbol con punteros far)

Layout de Nodo (punteros far = offset+segmento, 2 palabras cada uno): izquierdo en [BX+0] (offset)/ [BX+2] (segmento); derecho en [BX+4] (offset)/ [BX+6] (segmento); numero en [BX+8] — 10 bytes por nodo (no se usa numero en este ejercicio).

Protocolo: el llamador hace PUSH segmento_arbol; PUSH offset_arbol; CALL profundidad; POP profundidad — dos palabras de parámetro (puntero far) y una palabra de resultado; se deben conservar todos los registros.

⚠ DOS CORRECCIONES SOBRE EL CÓDIGO OFICIAL

```
profundidad proc
    PUSH BP
    MOV BP, SP
    PUSH BX
    PUSH ES
    MOV BX, SP:[BP + 6]      ; <-- "SP:" no es válido: SP no es un registro de segmento.
    Debe ser SS: (o nada, default)
    MOV ES, BX
    MOV BX, SP:[BP + 4]      ; <-- mismo problema
    PUSH AX
_if:
    CMP ES:[BX], 0          ; <-- BUG: esto compara el CONTENIDO apuntado por el puntero,
    no el
    JNE _else               ; puntero mismo. arbol==NULL se verifica sobre BX (el
    offset),
    XOR AX, AX              ; no leyendo memoria en esa dirección (que puede ser ba-
    sura si
    JMP _return             ; el puntero realmente es NULL/0).
```

- **SP:[BP+...]** — casi seguro es un error de tipeo/transcripción por **SS:[BP+...]** (SS es el único registro de segmento cuyo nombre empieza con S seguido de otra letra que tiene sentido acá; además el propio código, más abajo, sí usa **SS:[BP+6]** correctamente en el epílogo — inconsistencia que confirma que arriba falta la S). Como además **BP** ya usa **SS** por defecto, el prefijo ni siquiera es estrictamente necesario, pero si se escribe, tiene que ser un registro de segmento real.
- **CMP ES:[BX], 0** — la letra pide comprobar **arbol == NULL**. **arbol** es el puntero (par **ES:BX**), y la convención de todo el práctico (ver Ejercicio 3, índice 0 = “no hay nodo”) es representar el puntero nulo con offset 0. Comparar **BX** contra 0 es la comprobación correcta; comparar **ES:[BX]** contra 0 lee la **primera palabra del nodo apuntado** (que resulta ser **offset_izquierdo**, ver el layout) — una cosa totalmente distinta, y además una lectura de memoria inválida si el puntero de verdad fuera nulo.

CÓDIGO CORREGIDO

```

PUSH segmento_arbol
PUSH offset_arbol
CALL profundidad
POP profundidad

profundidad PROC
    PUSH BP
    MOV BP, SP                ; [BP+2]=retaddr(near,2B) [BP+4]=offset_arbol
    [BP+6]=segmento_arbol
    PUSH BX
    PUSH ES
    MOV BX, [BP+6]            ; segmento del árbol (SS por defecto, correcto: se lee de la
                                pila)
    MOV ES, BX
    MOV BX, [BP+4]            ; BX = offset del árbol (offset dentro de ES)
    PUSH AX

_if:
    CMP BX, 0                 ; (*) arbol == NULL se compara sobre el PUNTERO, no su
                                contenido
    JNE _else
    XOR AX, AX                ; profundidad(NULL) = 0
    JMP _return

_else:
    PUSH ES:[BX+2]            ; segmento de arbol->izquierdo
    PUSH ES:[BX]              ; offset de arbol->izquierdo
    CALL profundidad         ; profundidad(izquierdo), queda en el tope de la
                                pila
    POP AX                    ; AX <- profundidad izquierdo

    PUSH ES:[BX+6]            ; segmento de arbol->derecho
    PUSH ES:[BX+4]            ; offset de arbol->derecho
    CALL profundidad         ; (**) nombre corregido (el oficial tenía
                                "profundidad", typo)
    POP BX                    ; BX <- profundidad derecho

_max:
    CMP AX, BX
    JGE _endif
    MOV AX, BX
_endif:
    INC AX                    ; +1 por el nivel actual

_return:
    MOV SS:[BP+6], AX        ; escribimos el resultado en el slot de "segmento_arbol" (ya
                                no se necesita)
    MOV AX, SS:[BP+2]        ; AX <- dirección de retorno guardada

```

```
MOV  SS:[BP+4], AX      ; la reubicamos en el slot de "offset_arbol"
POP  AX
POP  ES
POP  BX
POP  BP
ADD  SP, 2              ; salta el slot viejo de "offset_arbol" (ya vacío de
uso)
RET                      ; retoma en la dirección reubicada; deja el resultado
arriba de todo
profundidad ENDP
```

(****) el PDF oficial escribe `CALL profundidad` en la segunda llamada recursiva — falta una “d”. Es un typo de tipeo (el ensamblador daría “símbolo indefinido”), no un error conceptual; queda corregido a `profundidad` arriba.

Verificación explícita de por qué el epílogo funciona (la técnica es correcta, solo tenía el bug de `_if` encima):

Llamando `SP0` al SP del llamador antes de `PUSH segmento_arbol`:

Paso	Acción	SP (relativo a SP0)
1	<code>PUSH segmento_arbol</code>	SP0-2
2	<code>PUSH offset_arbol</code>	SP0-4
3	<code>CALL profundidad</code> (apila retaddr, near)	SP0-6 ← esto es “SP_entry”
4	<code>PUSH BP</code> (BP←SP_entry-2)	SP0-8
5	<code>PUSH BX; PUSH ES; PUSH AX</code>	SP0-14
...	(cuerpo de la función, recursión balanceada: cada <code>PUSH</code> ×2 + <code>CALL</code> + <code>POP</code> ×1 neto es -2, se compensa entre las dos llamadas)	vuelve a SP0-14 antes de <code>_return</code>
6	<code>POP AX; POP ES; POP BX</code>	SP0-8
7	<code>POP BP</code> (SP = dirección de BP viejo + 2 = (SP_entry-2)+2 = SP_entry = SP0-6)	SP0-6
8	<code>ADD SP,2</code>	SP0-4
9	<code>RET</code> (desapila el retaddr reubicado en el paso <code>_return</code> , SP+=2)	SP0-2

Tras el `RET`, `SP=SP0-2`: queda **exactamente una palabra** pendiente en la pila del llamador — el resultado, listo para el `POP profundidad` del protocolo. Los 4 bytes de parámetros que entraron se convirtieron en los 2 bytes de resultado que salen, tal como exige la letra (“el resultado se devuelve en el stack y los argumentos deben retirarse del stack”). ✓ Verificado con `python -c "print(-6-8-14+14-6-6+4-2, 'debe dar -2')"` sobre los deltas de la tabla — cuadra.

C) CONSUMO DE STACK EN EL PEOR CASO, ÁRBOL DE N NODOS

Por activación: dirección de retorno (2, near) + parámetros (offset+segmento, 4) + `PUSH BP` (2) + `PUSH BX` (2) + `PUSH ES` (2) + `PUSH AX` (2) = **14 bytes**.

Igual que en el Ejercicio 3: el peor caso es una cadena degenerada de N nodos reales (árbol sin balancear, cada nodo con un solo hijo) — profundidad simultánea = N (nodos reales) + 1 (la activación final que entra con `arbol==NULL` y corta la recursión) = **N+1**.

Consumo máximo = $14 \times (N + 1)$ bytes

Verificación para N=2 (cadena de 2 nodos): `python -c "print(14*(2+1))"` → 42, coincide con la tabla de simulación de la sección Ejercicio 5 (misma técnica, per-activación distinto).

Ejercicio 5 — `cantidadBifurcaciones` (nodos con exactamente dos hijos)

No está en el material oficial de la cátedra — se resuelve siguiendo el mismo protocolo y estilo que el Ejercicio 4 (misma estructura `Nodo` con punteros `far`).

A) Alto nivel:


```
short cantidadBifurcaciones(Nodo* arbol) {
    short cant;
    if (arbol == NULL)
        cant = 0;
    else {
        short cIzq = cantidadBifurcaciones(arbol->izquierdo);
        short cDer = cantidadBifurcaciones(arbol->derecho);
        cant = cIzq + cDer;
        if (arbol->izquierdo != NULL && arbol->derecho != NULL)
            cant = cant + 1;    // este nodo es una bifurcación
    }
    return cant;
}
```

B) Assembler (mismo protocolo de invocación que el Ejercicio 4: `PUSH segmento; PUSH offset; CALL; POP resultado`):

```

cantidadBifurcaciones PROC
    PUSH BP
    MOV BP, SP                ; [BP+4]=offset_arbol [BP+6]=segmento_arbol
    PUSH BX
    PUSH CX
    PUSH ES
    MOV BX, [BP+6]
    MOV ES, BX
    MOV BX, [BP+4]            ; BX = offset del nodo actual (persiste durante toda la
activación)
    PUSH AX

    _if:
        CMP BX, 0
        JNE _else
        XOR AX, AX            ; arbol==NULL -> cant=0
        JMP _return

    _else:
        PUSH ES:[BX+2]        ; segmento de arbol->izquierdo
        PUSH ES:[BX]          ; offset de arbol->izquierdo
        CALL cantidadBifurcaciones ; BX sobrevive (esta misma función preserva BX
por contrato)
        POP AX                ; AX <- cIzq

        PUSH ES:[BX+6]        ; segmento de arbol->derecho
        PUSH ES:[BX+4]        ; offset de arbol->derecho
        CALL cantidadBifurcaciones
        POP CX                ; CX <- cDer

        ADD AX, CX            ; AX = cIzq + cDer

        CMP WORD PTR ES:[BX], 0 ; ¿arbol->izquierdo == NULL?
        JE _endif
        CMP WORD PTR ES:[BX+4], 0 ; ¿arbol->derecho == NULL?
        JE _endif
        INC AX                ; ambos hijos existen ->
bifurcación

    _endif:
    _return:
        MOV SS:[BP+6], AX
        MOV AX, SS:[BP+2]
        MOV SS:[BP+4], AX
        POP AX
        POP ES
        POP CX
        POP BX
        POP BP
        ADD SP, 2

```

```
RET
cantidadBifurcaciones ENDP
```

Por qué se necesita un registro más que en `profundidad` (`CX` además de `AX`, `BX`, `ES`): en `profundidad`, después de la segunda llamada recursiva ya no hace falta el puntero al nodo actual, así que `BX` se podía reciclar para guardar `pDer`. Acá, en cambio, el chequeo final (“¿ambos hijos existen?”) necesita **releer** `ES:[BX]` y `ES:[BX+4]` **después** de las dos llamadas recursivas — `BX` tiene que sobrevivir intacto hasta el final, así que `cIzq` y `cDer` necesitan cada uno su propio registro (`AX` y `CX`) en simultáneo.

C) CONSUMO DE STACK, PEOR CASO PARA ÁRBOL DE N NODOS

Por activación: $\text{retaddr}(2) + \text{parámetros}(4) + \text{BP}(2) + \text{BX}(2) + \text{CX}(2) + \text{ES}(2) + \text{AX}(2) = 16$ bytes. Mismo argumento de peor caso (cadena degenerada + activación final NULL): $\text{profundidad} = N+1$.

Consumo máximo = $16 \times (N + 1)$ bytes

Verificado: `python -c "for N in (1,2,3): print(N, 16*(N+1))"` → (1,32) (2,48) (3,64).

Ejercicio 6 — Torres de Hanoi

No está en el material oficial de la cátedra. Se resuelve con el protocolo indicado por la letra: `n` **viene en la pila**, y `moverDisco` “recibe los parámetros en el stack”. Como la letra no fija el orden exacto ni quién limpia, se adopta (y se deja explícita) la convención estándar de este práctico: **parámetros empujados de derecha a izquierda, y el llamador los retira tras el `CALL`** (igual que en `factorial` de la guía M6, y análoga a como cada `ADD SP,n` aparece en los ejercicios anteriores).

Alto nivel (dado por la letra):

```
void torres(short n, short i, short j) {  
    short k;  
    if (n == 1) {  
        moverDisco(i, j);  
    } else {  
        k = 6 - i - j;  
        torres(n-1, i, k);  
        torres(1, i, j);  
        torres(n-1, k, j);  
    }  
}
```

CÓDIGO

```

; void torres(short n, short i, short j)
; Convención: el llamador apila j, i, n (derecha a izquierda) y retira 6 bytes tras el
CALL.
; torres no devuelve valor. Se preservan AX, BX, CX.

torres PROC
    PUSH BP
    MOV  BP, SP                ; [BP+4]=n [BP+6]=i [BP+8]=j
    PUSH AX
    PUSH BX
    PUSH CX
    SUB  SP, 2                ; variable local k -> vive en [BP-8]

    MOV  AX, [BP+4]           ; AX = n
    CMP  AX, 1
    JNE  _else

_base:
    ; n==1 -> moverDisco(i, j)
    MOV  BX, [BP+8]           ; BX = j
    MOV  CX, [BP+6]           ; CX = i
    PUSH BX                   ; parámetro j (derecha primero)
    PUSH CX                   ; parámetro i
    CALL moverDisco
    ADD  SP, 4                ; torres retira los 4 bytes que ELLA
    empujó
    JMP  _return

_else:
    MOV  BX, 6
    SUB  BX, [BP+6]           ; BX = 6 - i
    SUB  BX, [BP+8]           ; BX = 6 - i - j
    MOV  [BP-8], BX           ; k = 6-i-j (variable local)

    ; torres(n-1, i, k)
    MOV  AX, [BP-8]           ; k
    PUSH AX
    MOV  AX, [BP+6]           ; i
    PUSH AX
    MOV  AX, [BP+4]           ; n
    DEC  AX
    PUSH AX
    CALL torres
    ADD  SP, 6

    ; torres(1, i, j)
    MOV  AX, [BP+8]           ; j
    PUSH AX
    MOV  AX, [BP+6]           ; i

```

```

    PUSH AX
    MOV  AX, 1
    PUSH AX
    CALL torres
    ADD  SP, 6

    ; torres(n-1, k, j)
    MOV  AX, [BP+8]                ; j
    PUSH AX
    MOV  AX, [BP-8]                ; k
    PUSH AX
    MOV  AX, [BP+4]                ; n
    DEC  AX
    PUSH AX
    CALL torres
    ADD  SP, 6

_return:
    ADD  SP, 2                    ; liberamos la variable local k
    POP  CX
    POP  BX
    POP  AX
    POP  BP
    RET                          ; el LLAMADOR retira n,i,j (6 bytes) de su propia pila
torres ENDP

```

Revisión de simetría PUSH/POP: PUSH BP/AX/BX/CX (4 registros) $\leftrightarrow$ POP CX/BX/AX/BP (orden inverso exacto) ✓. SUB SP,2 (reserva de k) $\leftrightarrow$ ADD SP,2 justo antes de empezar a desapilar registros ✓. Cada una de las 4 llamadas (moverDisco $\times 1$, torres $\times 3$) tiene su ADD SP,n de limpieza inmediatamente después del CALL correspondiente, con el tamaño exacto de lo que esa llamada empujó (4 para moverDisco, 6 para torres) ✓.

⚠ Supuesto explícito: la letra no dice si moverDisco retira ella misma sus parámetros o si lo hace el llamador, ni el orden de sus argumentos. Se asumió “llamador retira” (igual que para torres) porque es la convención dominante en todo el resto del práctico; si un enunciado de examen especifica lo contrario, sólo cambia el ADD SP,4 por nada (si moverDisco hiciera RET 4) — el resto del análisis no se ve afectado porque moverDisco es una rutina externa ya definida (caja negra) y no se conoce ni hace falta conocer su consumo interno de pila más allá de su propio CALL +parámetros.

B) CONSUMO DE STACK PARA N=3

Bytes por activación de torres: retaddr(2, near) + parámetros n,i,j (6) + BP (2) + AX (2) + BX (2) + CX (2) + variable local k (2) = **18 bytes**.

Profundidad: a diferencia de `busquedaBin` / `profundidad` (que llaman a dos subárboles y elegían la rama que llega más hondo), acá las tres llamadas de la rama `else` (`torres(n-1,i,k)`, `torres(1,i,j)`, `torres(n-1,k,j)`) son **secuenciales** — se ejecutan una después de la otra, nunca simultáneamente. Solo una está “viva” en la pila a la vez. La primera (`torres(n-1,i,k)`) es la que recursa más profundo, siguiendo la cadena `torres(n)→torres(n-1)→torres(n-2)→...→torres(1)` — exactamente **n** activaciones de `torres` simultáneas en el punto más profundo, y desde la más profunda (`torres(1)`, caso base) se dispara además una llamada transitoria a `moverDisco` (retaddr 2 + parámetros 4 = 6 bytes) antes de que nada se desapile.

Consumo máximo = $18 \times n + 6$ bytes

Para $n=3$: `python -c "print(18*3+6)"` → **60 bytes**.

Simulación explícita, $n=3$ (SP relativo, 0=antes de `PUSH n=3` por el programa principal):

Paso	Acción	SP acumulado
1	<code>PUSH j,i,n=3 + CALL torres(3,i,j)</code>	8
2	[dentro] <code>PUSH BP,AX,BX,CX + SUB SP,2 (k)</code>	18
3	<code>torres(3)</code> rama <code>else</code> : <code>PUSH k,i,n-1=2 + CALL torres(2,i,k)</code>	24
4	[dentro] <code>PUSH BP,AX,BX,CX + SUB SP,2</code>	34
5	<code>torres(2)</code> rama <code>else</code> : <code>PUSH k',i,n-1=1 + CALL torres(1,i,k')</code>	40
6	[dentro] <code>PUSH BP,AX,BX,CX + SUB SP,2</code>	50
7	<code>torres(1)</code> caso base: <code>PUSH j,i (params de moverDisco) + CALL moverDisco</code>	60

Pico = 60 bytes, exactamente cuando `moverDisco` arranca con las tres activaciones de `torres` ($n=3,2,1$) todavía abiertas debajo. Coincide con la fórmula $18 \cdot 3 + 6 = 60$. ✓

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica clave	Bytes/activación	Profundidad peor caso
1	Parámetro mixto registro+pila (BX + pila relativa a <code>ES</code> , no a <code>DS</code>); default de segmento con <code>BP</code> vs. con <code>[reg]</code> simple	— (no se pide)	—
2	Todo por registros (sin frame <code>BP</code>); pila usada solo para respaldar registros entre llamadas; profundidad = rama que llega antes a un caso base, no total de llamadas del árbol	2 (invocación) / 4 (c/u recursiva)	N-1 llamadas (N=5→18B)
3	Índice-a-dirección (<code>SHL x4</code>); reutilizar un registro (<code>SI</code>) como variable local que “sobrevive” la recursión por contrato de preservación de registros	6	N+1 (cadena degenerada)
4	Puntero <code>far</code> (offset+segmento, 2 palabras); “devolver resultado en la pila” reubicando la dirección de retorno un slot más abajo	14	N+1
5	Igual que 4, pero necesita un registro adicional porque el puntero al nodo debe sobrevivir hasta <i>después</i> de ambas llamadas recursivas	16	N+1
6	Tres llamadas recursivas <i>secuenciales</i> (no simultáneas) → la profundidad depende de la rama más honda de UNA sola, no de la suma de las tres; variable local (<code>k</code>) reservada con <code>SUB SP, n</code>	18 (torres) + 6 (moverDisco, transitorio)	n (torres)

Errores reales encontrados y corregidos en el material oficial de cátedra: Ejercicio 1 (2 prefijos de segmento invertidos), Ejercicio 3 (desbalance de pila + confusión dirección/valor — cambia el resultado numérico de 511 a 1536 bytes), Ejercicio 4 (condición de caso base mal formulada + typo de segmento `SP:` / `SS:` + typo de nombre `profundidad` / `profundidad`).

Para el examen

El patrón que se repite en los 6 ejercicios es siempre el mismo trámite, en este orden — memorízalo como checklist, no como fórmula:

1. **¿Cómo entran los parámetros?** ¿Registro o pila? Si es pila, ¿quién los retira (llamador o llamado)? Esto define si hace falta `PUSH BP; MOV BP, SP` (frame) o no.
2. **Armar el frame:** `PUSH BP; MOV BP, SP`, y ubicar cada parámetro en `[BP+4]`, `[BP+6]`, ... contando desde la dirección de retorno (2 bytes si el `CALL` es *near*, 4 si es *far*) hacia afuera.
3. **Salvar TODO registro de trabajo que vaya a sobrevivir una llamada recursiva** — con `PUSH` / `POP` simétricos, en orden LIFO exacto. Si un registro necesita sobrevivir *dos* llamadas recursivas seguidas (Ejercicio 5), necesita su propio slot — no se puede reciclar el mismo registro para las dos mitades del resultado.
4. **Elegir el segmento correcto en cada acceso a memoria:** por defecto `[BP+...]` usa `SS`, cualquier otro registro simple (`[BX]`, `[SI]`, `[DI]`) usa `DS` — y **cualquier dato en un segmento explícito distinto (`ES` en este práctico) necesita el prefijo escrito**, sin excepción. Los tres bugs reales que aparecieron en el material oficial de este práctico son, los tres, errores de este tipo — es, con diferencia, la fuente de errores más común y más fácil de arrastrar sin darse cuenta.
5. **Calcular el consumo de stack:** (dirección de retorno + registros salvados + parámetros + variables locales) × profundidad máxima **simultánea** de recursión — no la cantidad total de llamadas del árbol de recursión completo. Para funciones con más de una llamada recursiva, identificar si esas llamadas son **paralelas en el árbol de recursión** (como `fibonacci` / `profundidad` / `busquedaBin`, donde hay que analizar cuál rama llega primero a un caso base) o **secuenciales en el tiempo** (como Torres de Hanoi, donde solo una está activa a la vez y la profundidad depende de una sola cadena de llamadas, no de sumar las tres).